

CS-473: System Programming for Systems-on-Chip

Practical Work 2: The Mandelbrot Set

Lisa Suzuki (SCIPER: 408471)

Eduard Vlad (SCIPER: ??)

October 5, 2026

1 Introduction

This report documents two replacements for the reference `float` implementation (`fractal_float`):

1. `fractal_fxpt`, a signed Q8.24 fixed-point version;
2. `fractal_myflpt`, a custom 32-bit floating-point format with its own inlineable arithmetic library.

Both reach the same image as the reference, with $\leq 0.03\%$ of pixels differing, while eliminating all calls into `libgcc`'s soft-float library.

Experimental setup. Table 1 lists the configuration under which every figure in this report was obtained. All three programs share it exactly, so the comparisons that follow isolate the arithmetic representation and nothing else.

Table 1: Experimental setup.

Platform	
CPU	OR1300 (OpenRISC 1000 ISA, 5-stage pipeline, no FPU)
Instruction cache	8 kiB, direct mapped, FIFO replacement
Data cache	8 kiB, 4-way set associative, LRU, write-back
Toolchain	<code>or1k-elf-gcc 15.2.0</code>
Compiler flags	<code>-Os -DNDEBUG -D__OR1300__</code> (project default)
Workload	
Image size	512×512 pixels, <code>rgb565</code> frame buffer
Iteration limit	$n_{\max} = 64$
Viewport	$c_x \in [-2, 1), c_y \in [-1.5, 1.5)$
Coordinate step	$\Delta c = 3/512 = 0.005859375$
Total iterations	$\approx 3.87 \times 10^6$ (Section 4)
Origin of the reported figures	
Static code metrics	<code>or1k-elf-size</code> and <code>or1k-elf-objdump</code> on the ELF
Run-time metrics	OR1300 profiling counters (Section 5.2)
Numerical accuracy	host-side replay against IEEE-754 <code>double</code> (Section 4)

2 Fixed-point implementation

2.1 Format definition

Choosing m and f requires a bound on the largest intermediate value, which we derive here for this specific algorithm and viewport.

Let x_n, y_n be the values at the start of iteration n . The loop body computes $xx = x_n^2$, $yy = y_n^2$, updates x and y by the Mandelbrot recurrence, and only *then* tests $xx + yy < 4$. Consequently iteration $n + 1$ is reached only if $x_n^2 + y_n^2 < 4$. Under that hypothesis, and using $|c_x| \leq 2$, $|c_y| \leq 1.5$ for our viewport:

$$|x_{n+1}| = |x_n^2 - y_n^2 + c_x| \leq (x_n^2 + y_n^2) + |c_x| < 4 + 2 = 6, \quad (1)$$

$$|y_{n+1}| = |2x_n y_n + c_y| \leq (x_n^2 + y_n^2) + |c_y| < 4 + 1.5 = 5.5, \quad (2)$$

where the second line uses $|2xy| \leq x^2 + y^2$. Hence the largest value ever written to `xx + yy` is bounded by

$$x_{n+1}^2 + y_{n+1}^2 < 6^2 + 5.5^2 = 66.25. \quad (3)$$

(The very first pass, where $x_1 = c_x$ and $y_1 = c_y$, yields at most $4 + 2.25 = 6.25$ and is not critical.) The format must therefore represent magnitudes up to 66.25, so it needs $\lceil \log_2 66.25 \rceil = 7$ integer bits plus one sign bit. On a 32-bit integer this leaves exactly 24 fractional bits:

<code>fxpt = Q8.24,</code> range $[-128, 128)$, resolution $2^{-24} \approx 5.96 \times 10^{-8}$.
--

Q8.24 is the *tightest provably safe* choice: Q7.25 saturates at $64 < 66.25$ and is therefore not safe against bound (3), while Q4.28, which the comments in the skeleton's `main_flpt.c` suggest, saturates at 8 and would overflow badly. Instrumenting the algorithm over the whole image confirms the analysis: the largest value actually observed for `xx + yy` is $585\,942\,317 = 34.925$, i.e. 27 % of the available range, with **zero** overflow events in 3.87×10^6 iterations.

The declaration keeps the scale factor symbolic so the format can be re-tuned in one place:

```
1 #define FXPT_FRAC 24 // In Q(32-FXPT_FRAC).FXPT_FRAC
2 #define FX(x) ((fxpt)((x) * (1 << FXPT_FRAC))) // compile-time float -> fxpt conversion
3 typedef int32_t (fxpt);
```

Because `FX()` is a macro applied to literal constants, the conversions of `CX_0`, `CY_0` and `FRAC_WIDTH` are folded by the compiler and cost nothing at run time; no floating-point code survives in the binary.

2.2 Arithmetic operations

Addition and subtraction use the native 32-bit integer adder directly, both operands carrying the same scale (`x = xx - yy + cx`). Multiplication is the only operation needing care, since the Q16.48 product does not fit in 32 bits:

```
1 static inline fxpt mul(fxpt a, fxpt b) {
2     return (fxpt)((int64_t) a * b >> FXPT_FRAC); // Q8.24 * Q8.24 = Q16.48 -> >>24 -> Q8.24
3 }
```

This answers Hint 1 of the handout. What matters for our case is that the 8 discarded upper bits are always sign extension, which (3) proves, so the shift is lossless at the top. Truncation toward $-\infty$ costs at most one ulp (2^{-24}) per multiplication, which Section 4 shows to be harmless.

2.3 Limitations

Hint 2 of the handout asks how overflow and underflow are handled, given that the fixed-point type offers no detection. Our answer is to make them *impossible by construction* rather than to detect them at run time, which would cost a branch in the innermost loop:

- **Overflow** is excluded by the bound (3) together with the choice of Q8.24, as derived above. The same argument covers the intermediates: $|xx|, |yy| < 36$ and $|2xy| < 66$, all within $[-128, 128)$.
- **Underflow** is a graceful loss of precision, not a wrap-around: a product smaller than 2^{-24} simply truncates to zero. Since the escape test compares against 4.0, values that underflow are numerically irrelevant to the decision. The only visible effect is on pixels extremely close to the boundary of the set, quantified in Section 4.

Note that this guarantee is viewport-specific. A zoom feature would shrink Δc but leave the bound (3) unchanged (it depends only on the escape threshold and on $|c| \lesssim 2$), so Q8.24 would remain safe against overflow. What fails first is *resolution*, as discussed in Section 6.

3 Custom floating-point format: myflpt

3.1 Format definition

We adopted the layout suggested in the handout: a 32-bit word holding a sign bit, a 23-bit magnitude and an 8-bit exponent in excess-250.

bit 31	bits 30–8	bits 7–0
sign	magnitude m (23 bits, leading one <i>explicit</i>)	exponent e (excess-250)

A non-zero value is interpreted as

$$v = (-1)^s \cdot \frac{m}{2^{23}} \cdot 2^{e-250}, \quad \frac{1}{2} \leq \frac{m}{2^{23}} < 1, \quad (4)$$

so the magnitude is normalised to the interval $[0.5, 1)$ and bit 30 is always set for a non-zero number. The all-zero word `0x00000000` is the unique encoding of zero. Representable magnitudes span $2^{-251} \approx 2.8 \times 10^{-76}$ up to just below $2^5 = 32$.

Design rationale.

- **Exponent in the least significant bits, magnitude above it.** This ordering makes two operations cheap. First, an unsigned comparison of two same-sign words is already a correct magnitude comparison, so `order()` and `is_smaller()` reduce to integer compares. Second, doubling a value is a single increment of the exponent field.
- **Excess-250 rather than excess-127.** The algorithm only ever handles magnitudes below 66.25, and squaring drives values toward zero. Biasing the exponent so that the representable range is $[2^{-251}, 2^5)$ spends essentially all of the dynamic range *below* 1.0, where this algorithm needs it, instead of wasting half of it on values above 32 that the escape condition forbids. The bias also keeps the stored exponent unsigned, so no sign extension is needed when unpacking.
- **Explicit leading one instead of IEEE's hidden bit.** This removes the insert/extract logic from every pack and unpack. The price is exactly one bit of significand compared to `float`, quantified in Section 4.
- **Saturation and flush-to-zero instead of $\pm\infty$ and NaN.** There is no useful reaction to an infinity in this algorithm, and supporting the special encodings would add a test to every operation. Exponent overflow therefore returns the largest representable magnitude (≈ 32 , encoded `0x7FFFFFFF`) and exponent underflow returns zero. Section 3.3 shows why saturating at 32 cannot corrupt the escape test.

3.2 Arithmetic operations

All operators live in `include/myflpt.h` and are declared `static inline` so that they can be inlined into the hot loop, as recommended by the handout; Section 5.3 shows that this requires an explicit nudge to the compiler.

`float_to_myflpt()` converts an IEEE-754 `float` by re-biasing the exponent, since $1.m \cdot 2^{E-127} = 0.1m \cdot 2^{(E+124)-250}$, and by shifting the 24-bit significand $1.m$ right by one to place the leading one at bit 22 of the magnitude field. It is used only on compile-time constants and is fully folded away.

`add()` and `sub()` align, operate and normalise. `order()` first swaps the operands so that $|a| \geq |b|$, which guarantees that the exponent difference $d = e_a - e_b$ is non-negative and that the subtraction of magnitudes cannot go negative. The smaller magnitude is then shifted right by d and the two are added (equal signs) or subtracted (opposite signs), after which the result is re-normalised. Both mantissas are widened by `MYFLPT_G` = 8 guard bits before alignment, so that bits shifted out during alignment still contribute to the result; the guard bits are dropped at the very end. The guard $d > 23 + \text{MYFLPT_G}$ both short-circuits the case where b is negligible and avoids an undefined shift by more than 31. Subtraction is implemented as $a - b = a + (-b)$ by flipping the sign bit, which costs one XOR.

`mul()` adds the exponents ($e = e_a + e_b - 250$, the extra -250 undoing the double bias), XORs the signs, and forms the 46-bit product of the two 23-bit magnitudes in a `uint64_t`. Since both magnitudes lie in $[0.5, 1)$, the product lies in $[0.25, 1)$ and needs at most a single normalisation step, selected by one comparison against 2^{45} .

`is_smaller()` performs the escape test. Operands with different signs are ordered by sign alone; same-sign operands are ordered by the unsigned comparison of their (exponent, magnitude) fields, with the order reversed for two negative values.

The doubling trick. Computing $2xy$ deserves a remark. Because the exponent occupies the low bits of the word, doubling a value is simply $xy + 1$. We initially used that form, but it is unsafe: when `mul()` saturates, the exponent field is already `0xFF` and the increment carries into the magnitude and sign fields, producing `0x80000000`. We therefore use `two_xy = add(xy, xy)` instead, which saturates correctly. We verified that both forms produce a bit-identical image at this zoom level (0 of 262 144 pixels differ), because the saturating case only ever occurs on the iteration that escapes; the safe form was kept because the property does not survive a change of viewport.

3.3 Limitations

- **One bit less significant than float.** The explicit leading one costs one bit, and all operations truncate rather than round to nearest. Measured worst-case relative error is $2.4 \times 10^{-7} \approx 2^{-22}$ against $1.2 \times 10^{-7} = 2^{-23}$ for `float` (Table 2).
- **Magnitudes saturate at $2^5 = 32$.** Over the full image, `mul()` saturates 24 times, when it squares a value that has already escaped. This is harmless: the escape test compares against 4.0, and a saturated result of $32 > 4$ still fails the test, so the iteration terminates exactly as it should. A format with a wider exponent range would be required if the escape threshold were ever raised above 32.
- **No subnormals, infinities or NaNs.** `float_to_myflpt()` maps a subnormal input to zero. This is deliberate (see rationale above) and cannot be reached from our constant inputs.

4 Verification

Rather than judging the result by eye, both libraries were replayed on the host against an IEEE-754 `double` reference (`report/verify.c`), using the exact same source headers that are compiled for the OR1300. Three properties were checked: the accuracy of the individual operators, the ordering predicate, and the iteration count of every pixel of the final image.

Table 2: Worst-case relative error of the `myflpt` operators over 10^6 random operand pairs drawn from $[-4, 4]$.

Operation	worst relative error	$\approx$
<code>float_to_myflpt()</code>	1.192×10^{-7}	2^{-23}
<code>add()</code>	2.360×10^{-7}	2^{-22}
<code>sub()</code>	2.353×10^{-7}	2^{-22}
<code>mul()</code>	2.380×10^{-7}	2^{-22}
IEEE-754 <code>float</code> (for reference)	1.192×10^{-7}	2^{-23}

The bound of 2^{-22} for the arithmetic operators is exactly what the format predicts: the result magnitude can be as small as 0.5, so one truncated ulp of 2^{-23} represents a relative error of up to 2^{-22} . `is_smaller()` was checked exhaustively over all 81 ordered pairs drawn from $\{-8, -4, -1.5, -0.25, 0, 0.25, 1.5, 4, 8\}$ and agrees with the `double` ordering on all of them.

Table 3: Agreement of the full 512×512 image with a `double` reference evaluated at the same (decoded) coordinates. “Delta” is the difference in the returned iteration count.

Version	pixels differing	share	max. delta
<code>fxpt</code> (Q8.24)	39/262 144	0.0149 %	7
<code>myflpt</code>	66/262 144	0.0252 %	19

Both versions are therefore visually indistinguishable from the reference. The differing pixels all lie on the fractal boundary, where the iteration is chaotic and an error of one ulp in the input legitimately changes the escape iteration. This is a property of the Mandelbrot set, not a defect of the arithmetic. The total work is essentially identical in all three versions (3 873 537 iterations for `fxpt`, 3 873 492 for `myflpt`, 3 873 534 for the `double` reference, a spread of 0.002 %), which makes the performance comparison in the next section a fair one: the three programs execute the same algorithm the same number of times, and differ only in the cost of one iteration.

Finally, the accumulation `cx = cx + delta` repeated 512 times was checked for drift. Both formats land on exactly 1.0 with zero error, because $\Delta c = 3 \cdot 2^{-9}$ and the coordinates stay below 2 in magnitude, so every intermediate is exactly representable in both formats.

5 Performance

5.1 Static analysis

Table 4 summarises what the compiler produced. The decisive column is the last one: the number of helper-routine calls executed per Mandelbrot iteration.

Table 4: Static code metrics (`-Os`, `TARGET=OR1300`, including the profiling read-out code, which is identical in all four programs).

Version	.text [B]	instr. in calc...soft	helper calls per iteration
<code>fractal_flpt</code> (library float)	24 820	71	9
<code>fractal_fxpt</code> (Q8.24)	17 604	78	3
<code>fractal_myflpt</code>	18 236	89	8
<code>fractal_myflpt_always_inline</code>	20 216	560	3

The nine calls of the reference version are `4×__addsf3`, `3×__mulsf3`, `1×__subsf3` and `1×__lesf2`; the three calls of the fixed-point version are all `__muldi3`, emitted for the 64-bit intermediate product. Over the whole image this amounts to 34.9 million calls into libgcc for `fractal_flpt` against 11.6 million for `fractal_fxpt`.

The size of the called routines explains the rest. Table 5 gives the static instruction count of each helper, i.e. an upper bound on the path actually executed:

Table 5: Static size of the helper routines reached from the inner loop.

libgcc soft-float	instr.	our libraries	instr.
<code>__addsf3</code>	675	<code>myflpt add()</code>	82
<code>__subsf3</code>	637	<code>myflpt mul()</code>	63
<code>__mulsf3</code>	371	<code>__muldi3</code>	27
<code>__lesf2</code>	105		

5.2 Measurements on the virtual prototype

The four programs were instrumented with the OR1300 profiling module (`support/include/perf.h`). Counters are configured while the module is stopped, started immediately before `draw_fractal()` and stopped immediately after it, so that neither the frame-buffer clear nor the final `dcache_flush()` is included. The data-cache flush is performed after the measurement window.

Methodology. Each configuration was measured three times, with a board reset between runs so that every run starts from cold caches; re-running without a reset leaves the instruction cache warm and makes the second run look artificially fast. We report cycle counts rather than the `hh:mm:ss.mmm` read-out, because `perf_cycles_to_time()` divides by a CPU frequency expressed in kHz and therefore quantises the result to ± 1 ms.

The executed-instruction counter is deterministic for a given binary and serves as a cross-check that the three runs really performed the same work. The run-time, stall and bus-idle counters do vary slightly, because two bus masters are asynchronous to the program: the SDRAM controller issues an auto-refresh roughly every $78\ \mu\text{s}$ irrespective of CPU activity (`sdram.v`), and the graphics controller runs a DMA that re-reads the whole frame buffer once per display frame (`graphicsController.v`). Both loads are *common-mode*, identical in all four binaries, so they shift every row of Table 6 equally and do not distort the comparison. The display DMA in particular puts a floor under bus occupancy that has nothing to do with the arithmetic, which is worth keeping in mind when interpreting the bus-idle column.

Reproducibility. Across the three runs of each binary the executed-instruction count, the I\$ miss count and the D\$ miss count were *bit-identical*, confirming that every run performed exactly the same work. Run times agreed to within 0.0125% in the worst case (`fxpt`) and 0.0012% for `flpt`. The only counter with visible scatter is the I\$ miss penalty of the three fast binaries (1 586–8 002 cycles): with only 32–112 cold-start misses, the total is decided by whether each individual refill happened to collide with a display-DMA burst or an SDRAM refresh. At $< 0.001\%$ of run time that scatter is irrelevant, and we therefore do not

Table 6: Measured on the Gecko5 board. Median of three runs; raw UART logs in `report/measurement/`. The derived clock is $5\,971\,036\,224/100.524 = 59.40$ MHz and is identical for all four binaries.

Version	run time [s]	run time [cycles]	executed instructions	instr. per iteration	CPI	speed-up
<code>fractal_flpt</code>	100.524	5 971 036 224	3 292 407 183	850.0	1.814	1.00×
<code>fractal_fxpt</code>	8.853	525 909 811	479 408 457	123.8	1.097	11.35 ×
<code>fractal_myflpt</code>	39.391	2 339 826 624	2 039 215 915	526.5	1.147	2.55 ×
<code>.._always_inline</code>	30.738	1 825 845 296	1 632 802 044	421.5	1.118	3.27 ×

Table 7: Memory-system counters, median of three runs. Bus occupancy is $1 - (\text{bus idle}/\text{run time})$.

Version	I\$ misses	I\$ miss penalty [cycles]	% of run time	D\$ misses	bus occupancy
<code>fractal_flpt</code>	24 277 779	1 647 278 253	27.6 %	16 400	40.1 %
<code>fractal_fxpt</code>	32	1 586	< 0.001 %	16 389	28.2 %
<code>fractal_myflpt</code>	52	3 506	< 0.001 %	16 389	27.9 %
<code>.._always_inline</code>	112	7 685	< 0.001 %	16 390	27.9 %

quote those three values as precise figures. Only the `flpt` penalty, measured over 24.3 million events, is statistically meaningful.

Two independent confirmations that the counters are trustworthy. First, the derived clock frequency is 59.40 MHz for all four binaries, computed from four completely independent pairs of numbers. Second, the D-cache miss count is pinned by the frame buffer: it is $512 \times 512 \times 2 = 524\,288$ B and the D-cache line is 32 B (8 words, from the word index `s_stage2MemoryAddressReg[4:2]` in `d-cache.v`), which gives exactly 16 384 compulsory misses. The measured values are 16 389–16 400, i.e. the compulsory misses plus 5–16 for everything else in the program.

5.3 The cost of not inlining

The handout notes that in-lining the custom operators “also helps”. Declaring them `static inline` turns out *not* to be sufficient at the project’s default optimisation level: at `-Os` gcc keeps out-of-line copies of `add()` and `mul()` and calls them, so `fractal_myflpt` pays five calls to `add()` and three to `mul()` per iteration, which is 31.0 million calls over the image, almost as many as the reference version’s 34.9 million. We therefore built a second project, `fractal_myflpt_always_inline`, which differs from `fractal_myflpt` in exactly six lines: each `static inline` in `include/myflpt.h` carries an added `__attribute__((always_inline))`. The sources, makefile and instrumentation are byte-identical, so the comparison isolates the inlining decision.

Table 8: Effect of forcing inlining of the `myflpt` operators (measured, median of three runs).

Build	.text [B]	calls / iter.	instr. per iteration	run time [s]	I\$ misses
<code>static inline</code> only (<code>-Os</code>)	18 236	8	526.5	39.391	52
<code>+ always_inline</code>	20 216	3	421.5	30.738	112
<i>change</i>	+10.9 %	−5	−104.9	−22.0 %	+60

Forcing inlining removes all eight library calls, leaving only the three `__muldi3` calls that the 64-bit mantissa product requires, the same number the fixed-point version pays. The program runs $1.28\times$ faster.

The instruction count shows precisely where the gain comes from: $526.5 \rightarrow 421.5$ instructions per iteration, a saving of 104.9 over the eight eliminated calls, i.e. **13.1 instructions of call overhead per call**: the `l.jal`, its delay slot, the prologue register saves, the epilogue restores and the `l.jr`. A second, smaller gain is invisible in the instruction count: subtracting executed instructions, stall cycles and I\$ penalty from the run time leaves 261.3 million unattributed cycles for `fractal_myflpt` and 164.5 million for the inlined build. The difference, 96.8 million cycles over 31.0 million eliminated calls, is **3.12 cycles of branch penalty per call**, from the pipeline refills on the call and on the return.

Finally, the trade-off we expected to have to argue about did not materialise. The inlined build is 10.9 % larger, and its I\$ misses do rise from 52 to 112, but that costs 7 685 cycles in total, under 0.001 % of run time.

Even at 20 kiB of text the *hot loop* still fits comfortably in the 8 kiB instruction cache, so on this platform inlining is a pure win. That is a conclusion we could only reach by measuring: the course slides show the same trade-off going the other way once a program’s working set exceeds the cache.

6 Discussion

Where the speed-up really comes from. It is tempting to attribute the gain to “integer instead of floating point”. The measurements show a more precise, two-part picture. All four binaries execute the same algorithm over the same 3.87×10^6 iterations ($\pm 0.002\%$, Section 4), so every difference lies in the cost of one iteration.

The first part is instruction count: 850.0 instructions per iteration for the library `float`, against 123.8 for Q8.24 fixed point, a factor of 6.9. Table 5 shows why: `libgcc`’s `__addsf3` is 675 instructions of code generalised over cases this algorithm never produces, where a Q8.24 addition is one `l.add`.

The second part is the instruction cache, and it is the larger surprise. The reference version misses 24.3 million times and *waits 1.65 billion cycles for those refills, 27.6% of its entire run time*. The fixed-point version misses 32 times. Not 32 thousand: thirty-two, which is cold start and nothing else. Its hot loop (`calc_mandelbrot_point_soft` 78 instructions, `__muldi3` 27, `draw_fractal` 67) is a few hundred instructions and simply lives in the 8 kiB cache, whereas the soft-float version’s working set of four `libgcc` routines totalling nearly 1800 instructions plus the caller does not. This is visible in the CPI as well: 1.814 for `flpt` against 1.097 for `fxpt`, a core that is almost never waiting. The two effects compose into the measured 11.35 $\times$.

Fixed point or floating point? For this viewport the fixed-point version wins on every axis: 11.35 $\times$ against 3.27 $\times$ for the best custom-float build, a smaller binary, and marginally better accuracy (Table 3), because its 2^{-24} absolute resolution is finer than `myflpt`’s 2^{-22} relative resolution for values of order one, which is all this viewport contains. The custom format nevertheless meets the handout’s requirement comfortably, at 2.55 $\times$ faster than the built-in library as written and 3.27 $\times$ with `always_inline`, and it earns its place as soon as the dynamic range stops being known in advance. With a zoom feature Δc shrinks geometrically while the escape bound stays at 4.0; Q8.24 keeps its 24 fractional bits regardless and becomes visibly quantised after roughly 2^{20} magnification, whereas `myflpt` redistributes precision automatically. The eight integer bits that Q8.24 must reserve for the escape excursion derived in (3) are pure overhead at high zoom, which is exactly where a floating-point representation pays for itself.

What is left on the table. All four binaries still call `__muldi3` three times per iteration to form a 64-bit product from 32-bit operands; the OR1300 has a $32 \times 32 \rightarrow 32$ multiplier (`l.mul`) but no widening multiply, so `gcc` falls back on the library routine. Splitting the operands into 16-bit halves by hand, or narrowing the `myflpt` magnitude field to 16 bits so a single `l.mul` suffices, would remove the last call; the second option trades roughly seven bits of significand, which Table 2 suggests the image would survive. We did not implement either.

A second limit is the bus. Even the fixed-point version, which barely touches memory for instructions, shows 28.2% bus occupancy. Much of that floor is not ours: the graphics controller re-reads the whole 512 kiB frame buffer once per display frame throughout the measurement. It is identical in all four binaries and therefore does not distort the comparison, but it means the bus-occupancy column should not be read as a property of the arithmetic alone.

Honest limits of the breakdown. Executed instructions, stall cycles and I\$ penalty do not add up to the run time: 7.1% of `fxpt`’s cycles and 9.0% of the inlined `myflpt`’s remain unattributed. The controlled inlining experiment identifies branch penalty as part of it (3.12 cycles per call, Section 5.3), and the remainder is loop branches, D-cache write stalls and bus contention. We did not enable `PERF_BRANCH_PENALTY_MASK`, so we present this as a partial, not a complete, cycle accounting.

7 Conclusion

Both required implementations are complete, validated and measured on the virtual prototype. The fixed-point version uses Q8.24, a choice justified by deriving a hard bound of 66.25 on the largest intermediate value and confirmed by measuring a maximum of 34.9 over 3.87 million iterations with zero overflows. The custom floating-point format uses an explicit leading one and an excess-250 exponent, deliberately omitting subnormals, infinities, NaNs and round-to-nearest in exchange for an `add()` eight times smaller than `libgcc`’s.

Both reproduce the reference image to within 0.03% of pixels, all on the fractal boundary, and both eliminate every soft-float call from the binary.

On the board, the fixed-point version runs **11.35**× faster than the library `float` and the custom floating-point version **2.55**× faster, rising to **3.27**× once the operators are genuinely inlined, so the handout’s requirement that the custom format beat the built-in library is met with margin.

Two findings were only reachable by measuring on the target. The first is that the speed-up is not mainly about arithmetic: 6.9× of it comes from executing fewer instructions, but the rest comes from the instruction cache, where the reference version spends 27.6% of its run time waiting for refills after 24.3 million misses while the fixed-point version misses 32 times in total. The second is that `static inline` does not inline at -Os: the custom library was paying eight function calls per iteration, at 13.1 wasted instructions and 3.12 cycles of branch penalty each, until `always_inline` was applied, for 10.9% more code and no measurable cache cost. Neither of these is visible in the source, and neither could have been predicted from first principles; both came out of the profiling counters. That, rather than the Mandelbrot set, is what this practical work teaches.